

LAPORAN PRAKTIKUM
KONSTRUKSI DAN EVOLUSI PERANGKAT LUNAK
PERTEMUAN 4
CI/CD UNTUK VUE.JS



Disusun oleh:

Nama : Hafidz Rizqullah Prasetya
NIM : 24/535493/SV/24243
Kelas : PL5A1
Dosen Pengampu : Galih Malela Damaraji

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Isi	2
Tujuan Praktikum	3
Dasar Teori	4
2.1 Single Page Application dan Vue 3	4
2.2 Memisahkan Alamat API dari Kode	4
2.3 CORS pada API Laravel	5
2.4 Pipeline Frontend Empat Job	5
2.5 Pengujian Unit dengan Vitest	5
Hasil dan Pembahasan	7
3.1 Alat dan Bahan	7
3.2 Endpoint JSON di Laravel	7
3.2.1 Mengizinkan Akses dari Vue	8
3.3 Aplikasi Vue 3	9
3.3.1 Alamat API dari Variabel Lingkungan	10
3.4 Workflow Frontend Empat Job	10
3.4.1 Deploy Dilewati pada Pull Request	13
3.4.2 Isi Direktori dist di Job Deploy	14
3.5 Unit Test Vitest	15
3.6 Bukti Pipeline Berhenti Saat Test Gagal	16
Kesimpulan	18
Daftar Pustaka	19

Tujuan Praktikum

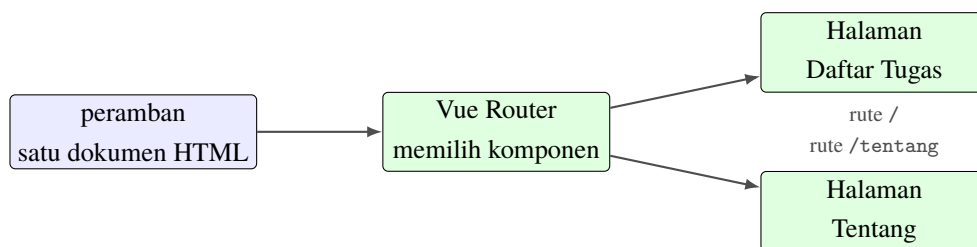
Sesuai arahan pada pertemuan keempat dan deskripsi tugas di Google Classroom *Konstruksi dan Evolusi Perangkat Lunak 2026*, tujuan praktikum ini adalah:

1. Mahasiswa mampu menyediakan satu endpoint JSON dari tabel CRUD pada Laravel dan memastikan aplikasi Vue dapat mengaksesnya.
2. Mahasiswa mampu membuat aplikasi Vue 3 dengan minimal dua halaman ber-router yang mengambil alamat API dari variabel lingkungan.
3. Mahasiswa mampu menyusun workflow empat job untuk frontend (lint, test, build, deploy) yang dirangkai dengan `needs`: dan memakai `npm ci` serta `cache npm`.
4. Mahasiswa mampu mengoper hasil build ke job deploy tanpa menjalankan build ulang, dan menampilkan isi direktori `dist`.
5. Mahasiswa mampu menulis minimal satu unit test Vitest yang lulus di GitHub Actions tanpa Laravel berjalan.
6. Mahasiswa mampu membuktikan bahwa Pull Request melewati tahap deploy dan pipeline berhenti ketika satu test sengaja digagalkan.

Dasar Teori

2.1 Single Page Application dan Vue 3

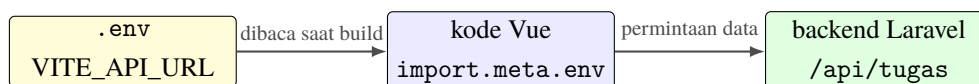
Vue 3 adalah kerangka kerja JavaScript untuk membangun antarmuka yang reaktif [1]. Pada aplikasi satu halaman, perpindahan antar halaman tidak memuat ulang dokumen, melainkan ditangani oleh *router* di sisi klien. Vue Router memetakan alamat ke komponen sehingga satu berkas HTML dapat melayani beberapa halaman [2].



Gambar 1: Pada aplikasi satu halaman, Vue Router memetakan alamat ke komponen tanpa memuat ulang dokumen.

2.2 Memisahkan Alamat API dari Kode

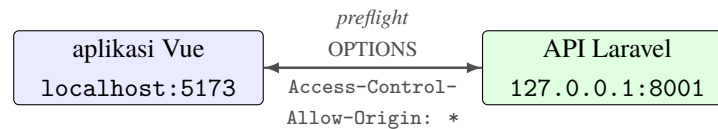
Menuliskan alamat API langsung di dalam kode menyulitkan perpindahan antara lingkungan lokal dan produksi. Vite menyediakan variabel lingkungan berawalan `VITE_` yang dibaca saat build dan diakses lewat `import.meta.env` [3]. Dengan pola ini, alamat backend cukup diganti di berkas `.env` tanpa mengubah kode sumber.



Gambar 2: Alamat API diambil dari berkas `.env`, sehingga alamat backend bisa diganti tanpa mengubah kode.

2.3 CORS pada API Laravel

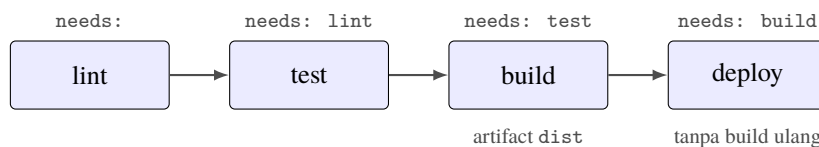
CORS mengatur izin akses lintas domain di peramban [4]. Ketika aplikasi Vue dijalankan di `localhost:5173` dan API berada di `127.0.0.1:8001`, keduanya dianggap domain berbeda. Laravel mengatur izin itu lewat berkas `config/cors.php` yang menentukan jalur, metode, dan asal yang diizinkan [5].



Gambar 3: Peramban mengirim permintaan *preflight* sebelum mengambil data. Server menjawab dengan header izin CORS.

2.4 Pipeline Frontend Empat Job

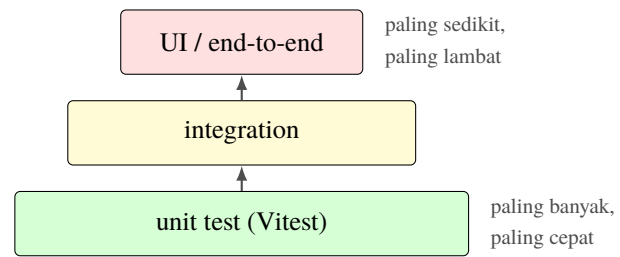
Workflow frontend mengikuti pola yang sama dengan pipeline backend: setiap job berjalan di mesin bersih dan dirangkai memakai `needs`: [6]. Pemasangan dependensi memakai `npm ci` agar versi paket persis mengikuti `package-lock.json`, dan cache npm mempercepat pemasangan pada job berikutnya [7]. Hasil `npm run build` diunggah sebagai artifact, lalu diunduh job `deploy`. Karena artifact sudah berisi direktori `dist`, job `deploy` tidak perlu menjalankan `npm run build` lagi [8].



Gambar 4: Empat job frontend dirangkai berurutan. Hasil build dioper ke job `deploy` lewat artifact.

2.5 Pengujian Unit dengan Vitest

Vitest menjalankan pengujian unit untuk proyek Vite [9]. Karena pengujian dijalankan di lingkungan Node tanpa backend, logika yang diuji harus mandiri dan tidak bergantung pada Laravel. Pada praktikum ini saya memisahkan logika pemformatan dan peringkasan ke berkas `src/lib/format.js` supaya dapat diuji tanpa menjalankan server.



Gambar 5: Piramida pengujian: unit test berada di lapisan terbawah dan paling banyak jumlahnya. Vitest dipakai pada lapisan ini.

Hasil dan Pembahasan

Pada bab ini saya menjelaskan pekerjaan untuk Tugas 3 sesuai instruksi di Classroom. Saya melanjutkan repository `evolusi-pl-535493` dengan menambahkan endpoint JSON di sisi Laravel dan aplikasi Vue 3 di folder `frontend/`.

3.1 Alat dan Bahan

Alat dan bahan yang digunakan:

- Laptop dengan OS Linux, akses internet, dan akun GitHub `hafidzrizqullahprasetya`.
- PHP 8.5.10, Composer 2.10.3, Laravel 13, Node.js 26, npm 11, dan Git.
- Repository: `hafidzrizqullahprasetya/evolusi-pl-535493` (publik) dengan aplikasi Laravel dari pertemuan sebelumnya.
- Google Classroom: kelas *Konstruksi dan Evolusi Perangkat Lunak 2026* [875340560422], tugas *Pengumpulan Tugas 3* dengan tenggat 28 September 2026.

3.2 Endpoint JSON di Laravel

Saya menambahkan tabel tugas beserta model dan pengendali untuk membacanya. Endpoint yang tersedia ada dua: daftar seluruh tugas dan detail satu tugas. Berkas `routes/api.php` mendaftarkan keduanya, lalu `bootstrap/app.php` diarahkan agar memuat berkas rute api tersebut.

```
1 class TugasController extends Controller
2 {
3     public function index(): JsonResponse
4     {
5         return response()->json([
6             'data' => Tugas::query()->orderBy('tenggat')->get(),
7         ]);
```

```

8   }
9 }

```

Listing .1: Pengendali daftar tugas.

Hasil pemanggilan endpoint lewat curl terlihat pada Gambar 1. Respons berisi empat tugas contoh dengan kolom judul, mata_kuliah, tenggat, dan selesai.

```

hafidzprasetya@hafidzprasetya:~/Project/evolusi-pl-535493$ curl -s http://127.0.0.1:8001/api/tugas | python3 -m json.tool
{
  "data": [
    {
      "id": 6,
      "judul": "Laporan Praktikum P3",
      "mata_kuliah": "PPD",
      "tenggat": "2026-09-25T00:00:00.000000Z",
      "selesai": true,
      "created_at": "2026-09-17T09:08:22.000000Z",
      "updated_at": "2026-09-17T09:08:22.000000Z"
    },
    {
      "id": 5,
      "judul": "Pengumpulan Tugas 3",
      "mata_kuliah": "KEPL",
      "tenggat": "2026-09-28T00:00:00.000000Z",
      "selesai": false,
      "created_at": "2026-09-17T09:08:22.000000Z",
      "updated_at": "2026-09-17T09:08:22.000000Z"
    },
    {
      "id": 8,
      "judul": "Resume Integrasi Nasional",
      "mata_kuliah": "PPKN",
      "tenggat": "2026-09-30T00:00:00.000000Z",
      "selesai": true,
      "created_at": "2026-09-17T09:08:22.000000Z",
      "updated_at": "2026-09-17T09:08:22.000000Z"
    },
    {
      "id": 7,
      "judul": "Proyek Kelompok Game",
      "mata_kuliah": "PPG",
      "tenggat": "2026-10-02T00:00:00.000000Z",
      "selesai": false,
      "created_at": "2026-09-17T09:08:22.000000Z",
      "updated_at": "2026-09-17T09:08:22.000000Z"
    }
  ]
}

```

Gambar 1: Keluaran curl ke /api/tugas dalam format JSON.

3.2.1 Mengizinkan Akses dari Vue

Karena aplikasi Vue berjalan di alamat yang berbeda, saya membuka CORS khusus untuk rute api. Saya menjalankan `php artisan config:publish cors` untuk menerbitkan berkas konfigurasinya, lalu menyesuaikan jalur, metode, dan asalnya. Pengujian dengan permintaan *preflight* menunjukkan header yang diharapkan:

```

1 curl -s -i http://127.0.0.1:8001/api/tugas | grep -iE 'HTTP/|content-type|access-
   ↪ control'
2 # HTTP/1.1 200 OK
3 # Content-Type: application/json
4 # Access-Control-Allow-Origin: *

```

Listing .2: Pemeriksaan CORS pada endpoint.

```
hafidzprasetya@hafidzprasetya:~/Project/evolusi-pl-535493$ curl -s -i http://127.0.0.1:8001/api/tugas | grep -iE 'HTTP/[
content-type|access-control'
HTTP/1.1 200 OK
Content-Type: application/json
Access-Control-Allow-Origin: *
```

Gambar 2: Respons endpoint yang menyertakan header Access-Control-Allow-Origin.

3.3 Aplikasi Vue 3

Aplikasi Vue saya letakkan di folder frontend/. Strukturnya memisahkan halaman, router, dan logika yang bisa diuji:

```
1 frontend/
2   index.html
3   package.json
4   vite.config.js
5   eslint.config.js
6   src/
7     main.js
8     App.vue
9     router/index.js
10    lib/format.js
11    lib/api.js
12    views/DaftarTugas.vue
13    views/Tentang.vue
14    tests/format.test.js
```

Listing .3: Struktur folder frontend.

Ada dua halaman ber-router. Halaman DaftarTugas.vue menampilkan data dari Laravel, sedangkan Tentang.vue menjelaskan asal alamat API. Router memetakan keduanya di src/router/index.js:

```
1 const routes = [
2   { path: '/', name: 'tugas', component: DaftarTugas },
3   { path: '/tentang', name: 'tentang', component: Tentang },
4 ]
```

Listing .4: Pemetaan rute halaman.

3.3.1 Alamat API dari Variabel Lingkungan

Alamat API tidak ditulis di dalam komponen. Fungsi `alamatApi` membacanya dari `import.meta.env.VITE_API_URL`, dengan nilai cadangan `/api` bila variabelnya kosong:

```
1 export function alamatApi(dasar) {
2   const nilai = dasar ?? import.meta.env.VITE_API_URL
3
4   if (!nilai) {
5     return '/api'
6   }
7
8   return nilai.replace(/\/+$/, '')
9 }
```

Listing .5: Pembacaan alamat API dari variabel lingkungan.

Nilai variabelnya dicatat pada `frontend/.env.example` sebagai `VITE_API_URL=http://127.0.0.1:8000/api`, sehingga pengguna lain cukup menyalin berkas itu menjadi `.env` dan menyesuaikan alamatnya.

3.4 Workflow Frontend Empat Job

Workflow `.github/workflows/frontend.yml` berisi empat job berurutan: `lint`, `test`, `build`, dan `deploy`. Job `lint` dan `test` memakai `npm ci` dengan cache `npm` yang diarahkan ke `frontend/package-lock.json`. Pemeriksaan gaya kode pada job `lint` memakai `ESLint` [10]. Job `build` mengunggah direktori `frontend/dist` sebagai artifact, dan job `deploy` mengunduh artifact itu tanpa menjalankan `npm run build` lagi.

```
1 name: Frontend
2
3 on:
4   push:
5     branches: ['**']
6     paths: ['frontend/**', '.github/workflows/frontend.yml']
7   pull_request:
8     paths: ['frontend/**', '.github/workflows/frontend.yml']
9
10 jobs:
11   lint:
12     name: Lint
13     runs-on: ubuntu-latest
14     steps:
```

```

15     - name: Ambil kode
16       uses: actions/checkout@v4
17
18     - name: Siapkan Node
19       uses: actions/setup-node@v4
20       with:
21         node-version: '22'
22         cache: 'npm'
23         cache-dependency-path: frontend/package-lock.json
24
25     - name: Pasang dependensi
26       working-directory: frontend
27       run: npm ci
28
29     - name: Periksa gaya kode
30       working-directory: frontend
31       run: npm run lint
32
33   test:
34     name: Test
35     needs: lint
36     runs-on: ubuntu-latest
37     steps:
38       - name: Ambil kode
39         uses: actions/checkout@v4
40
41       - name: Siapkan Node
42         uses: actions/setup-node@v4
43         with:
44           node-version: '22'
45           cache: 'npm'
46           cache-dependency-path: frontend/package-lock.json
47
48       - name: Pasang dependensi
49         working-directory: frontend
50         run: npm ci
51
52       - name: Jalankan pengujian
53         working-directory: frontend
54         run: npm test
55
56   build:
57     name: Build
58     needs: test
59     runs-on: ubuntu-latest
60     steps:
61       - name: Ambil kode
62         uses: actions/checkout@v4

```

```

63
64     - name: Siapkan Node
65       uses: actions/setup-node@v4
66       with:
67         node-version: '22'
68         cache: 'npm'
69         cache-dependency-path: frontend/package-lock.json
70
71     - name: Pasang dependensi
72       working-directory: frontend
73       run: npm ci
74
75     - name: Bangun aplikasi
76       working-directory: frontend
77       run: npm run build
78
79     - name: Simpan hasil build
80       uses: actions/upload-artifact@v4
81       with:
82         name: dist
83         path: frontend/dist
84         retention-days: 1
85
86   deploy:
87     name: Deploy
88     needs: build
89     if: github.ref == 'refs/heads/main' && github.event_name == 'push'
90     runs-on: ubuntu-latest
91     environment: production
92     steps:
93       - name: Ambil hasil build
94         uses: actions/download-artifact@v4
95         with:
96           name: dist
97           path: dist
98
99       - name: Tampilkan isi dist
100        run: |
101          ls -R dist
102          echo "--- isi index.html ---"
103          cat dist/index.html

```

Listing .6: Workflow empat job pada frontend.yml.

Pada run terakhir di main, keempat job hijau setelah deploy produksi disetujui:

```

1 gh run view 35203079450 --json jobs --jq '.jobs[] | "\(.name): \(.conclusion)'"
2 # Lint: success
3 # Test: success

```

```
4 # Build: success
5 # Deploy: success
```

Listing .7: Keempat job frontend hijau.



```
hafidzprasetya@hafidzprasetya:~/Project/evolusi-pl-535493$ gh run view 35203079450 --json jobs --jq '.jobs[] | \"(.name): \"(.conclusion)\"'
Working...
Working...
Lint: success
Test: success
Build: success
Deploy: success
```

Gambar 3: Keempat job workflow frontend hijau pada run di main.

3.4.1 Deploy Dilewati pada Pull Request

Sama seperti pipeline backend, job deploy diberi syarat hanya berjalan dari branch main pada kejadian push. Karena itu, ketika pekerjaan berada di branch fitur atau Pull Request, job deploy terlewat sementara lint, test, dan build tetap berjalan:

```
1 gh run view 35202752695 --json jobs --jq '.jobs[] | \"(.name): \"(.conclusion)\"'
2 # Lint: success
3 # Test: success
4 # Build: success
5 # Deploy: skipped
```

Listing .8: Deploy terlewat pada push ke branch fitur.

```
hafidzprasetya@hafidzprasetya:~/Project/evolusi-pl-535493$ gh run view 35202752695 --json jobs --jq '.jobs[]|"{.name}:  
\\(.conclusion)"}'  
Working...  
Working...  
Lint: success  
Test: success  
Build: success  
Deploy: skipped
```

Gambar 4: Push ke branch fitur menjalankan lint, test, dan build, sementara deploy terlewat.

3.4.2 Isi Direktori dist di Job Deploy

Job deploy menampilkan isi direktori dist ke log supaya terlihat bahwa berkas hasil build benar-benar sampai ke job itu. Artifact diunduh lebih dulu, lalu isinya dicetak:

```
1 - name: Ambil hasil build  
2   uses: actions/download-artifact@v4  
3   with:  
4     name: dist  
5     path: dist  
6 - name: Tampilkan isi dist  
7   run: |  
8     ls -R dist  
9     echo "--- isi index.html ---"  
10    cat dist/index.html
```

Listing .9: Langkah job deploy.

```

hafidzprasetya@hafidzprasetya:~/Project/evolusi-pl-535493$ gh run view --job=105142471864 --log | grep -E 'Z (\./|dist|total|assets|index|[-rw])' | head -20
Deploy Ambil hasil build      2026-09-17T09:05:58.4426122Z with:
Deploy Ambil hasil build      2026-09-17T09:05:58.7285123Z - dist (ID: 10488014290, Size: 36673, Expected Digest: sha256:22b8c1425211c1af86a15a4c78673253c62dcc73238b6ee2ad77fcf499ca2066)
Deploy Tampilkan isi dist     2026-09-17T09:05:58.9350899Z dist:
Deploy Tampilkan isi dist     2026-09-17T09:05:58.9351763Z assets
Deploy Tampilkan isi dist     2026-09-17T09:05:58.9352642Z index.html
Deploy Tampilkan isi dist     2026-09-17T09:05:58.9353540Z dist/assets:
Deploy Tampilkan isi dist     2026-09-17T09:05:58.9354414Z index-D6bn5oyt.js
Deploy Tampilkan isi dist     2026-09-17T09:05:58.9355350Z index-uFBzDM29.css
Deploy Tampilkan isi dist     2026-09-17T09:05:58.9355987Z --- isi index.html ---

```

Gambar 5: Log job Deploy yang menampilkan isi direktori dist hasil build.

3.5 Unit Test Vitest

Logika pemformatan tanggal dan peringkasan tugas saya taruh di `src/lib/format.js` agar bisa diuji tanpa backend. Berkas `tests/format.test.js` menguji fungsi tersebut dengan tujuh kasus, termasuk tanggal kosong, tanggal tidak valid, dan daftar tugas kosong. Pengujian dijalankan dengan `npm test`:

```

1 npm test
2 # Test Files 1 passed (1)
3 #      Tests 7 passed (7)

```

Listing .10: Hasil pengujian Vitest di laptop.

```
> kepl-frontend@0.1.0 test
> vitest run

RUN v3.2.7 /home/hafidzprasetya/Project/evolusi-pl-535493/frontend
✓ tests/format.test.js (7 tests) 16ms
  ✓ formatTanggal > mengubah tanggal ISO menjadi teks Indonesia 13ms
  ✓ formatTanggal > mengembalikan teks cadangan untuk tanggal kosong 0ms
  ✓ formatTanggal > mengembalikan teks cadangan untuk tanggal tidak valid 0ms
  ✓ ringkasan > menghitung jumlah tugas selesai dan belum 1ms
  ✓ ringkasan > mengembalikan nol untuk daftar kosong 0ms
  ✓ alamatApi > membuang garis miring di akhir 0ms
  ✓ alamatApi > memakai nilai bawaan bila alamat kosong 0ms

Test Files 1 passed (1)
Tests 7 passed (7)
Start at 16:26:03
Duration 270ms (transform 24ms, setup 0ms, collect 19ms, tests 16ms, environment 0ms, prepare 65ms)

rendering chunks...
computing gzip size...
dist/index.html 0.40 kB | gzip: 0.27 kB
dist/assets/index-uFBzDM29.css 0.43 kB | gzip: 0.28 kB
dist/assets/index-D6bn5oyt.js 91.53 kB | gzip: 35.75 kB
✓ built in 546ms
```

Gambar 6: Pemeriksaan gaya kode, pengujian Vitest, dan build yang dijalankan di laptop.

3.6 Bukti Pipeline Berhenti Saat Test Gagal

Sesuai instruksi tugas, saya sengaja menggagalkan satu test untuk membuktikan pipeline berhenti. Nilai harapan pada kasus uji peringkasan saya ubah dari 2 menjadi 9, lalu push. Job test gagal, dan job build serta deploy otomatis terlewati karena needs: tidak terpenuhi.

```
1 gh run view 35202565338 --json jobs --jq '.jobs[] | "\(.name): \(.conclusion)'"
2 # Lint: success
3 # Test: failure
4 # Build: skipped
5 # Deploy: skipped
6
7 # expected { total: 3, selesai: 2, belum: 1 } to deeply equal
8 #   { total: 3, selesai: 9, belum: 1 }
9 # Tests 1 failed | 6 passed (7)
```

Listing .11: Pipeline frontend merah pada test yang sengaja digagalkan.

```

hafidzprasetya@hafidzprasetya:~/Project/evolusi-pl-535493$ gh run view 35202565338 --json jobs --jq '.jobs[] | "\(.name): \(.conclusion)"; echo; gh run view 35202565338 --log-failed | sed -r 's/\^\^[0-9;]*m//g' | grep -E 'FAIL|expected|Test Files|Tests ' | sed -E 's/^\.*Z //'
Working...
Working...
Lint: success
Test: failure
Build: skipped
Deploy: skipped

→ expected { total: 3, selesai: 2, belum: 1 } to deeply equal { total: 3, selesai: 9, belum: 1 }
----- Failed Tests 1 -----
FAIL tests/format.test.js > ringkasan > menghitung jumlah tugas selesai dan belum
AssertionError: expected { total: 3, selesai: 2, belum: 1 } to deeply equal { total: 3, selesai: 9, belum: 1 }
Test Files 1 failed (1)
Tests 1 failed | 6 passed (7)
##[error]AssertionError: expected { total: 3, selesai: 2, belum: 1 } to deeply equal { total: 3, selesai: 9, belum: 1 }

```

Gambar 7: Satu test Vitest yang digagalkan menghentikan pipeline: build dan deploy terlewat.

Setelah bukti itu tersimpan, saya mengembalikan nilai harapan ke 2 dan push lagi. Pipeline kembali hijau pada commit berikutnya.

Kesimpulan

Berdasarkan praktikum pada Pertemuan 4, dapat disimpulkan beberapa hal:

1. Endpoint JSON `/api/tugas` berhasil disediakan dan dapat diakses aplikasi Vue setelah CORS dibuka untuk rute api.
2. Aplikasi Vue 3 di folder `frontend/` memiliki dua halaman ber-router, dan alamat API diambil dari `VITE_API_URL` alih-alih ditulis di dalam kode.
3. Workflow frontend berisi empat job (`lint`, `test`, `build`, `deploy`) yang dirangkai dengan `needs`: dan memakai `npm ci` beserta `cache npm`.
4. Hasil build dioper ke job `deploy` lewat artifact, dan job `deploy` tidak menjalankan build ulang serta menampilkan isi direktori `dist` ke log.
5. Deploy terbukti hanya berjalan dari `main`; pada Pull Request dan branch fitur, job itu terlewati.
6. Pipeline terbukti berhenti ketika satu test Vitest sengaja digagalkan, lalu kembali hijau setelah kesalahan diperbaiki.

Dengan ini alur CI/CD repository `evolusi-pl-535493` sudah mencakup sisi backend dan frontend, dengan deploy yang terjaga pada branch `main`.

Daftar Pustaka

- [1] Vue.js, “Vue.js Documentation.” [Online]. Tersedia: <https://vuejs.org/guide/introduction.html>.
- [2] Vue.js, “Vue Router.” [Online]. Tersedia: <https://router.vuejs.org/>.
- [3] Vite, “Env Variables and Modes.” [Online]. Tersedia: <https://vite.dev/guide/env-and-mode>.
- [4] MDN Web Docs, “Cross-Origin Resource Sharing (CORS).” [Online]. Tersedia: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>.
- [5] Laravel, “Laravel Documentation.” [Online]. Tersedia: <https://laravel.com/docs>.
- [6] GitHub Docs, “Understanding GitHub Actions.” [Online]. Tersedia: <https://docs.github.com/en/actions/learn-github-actions/understanding-github-actions>.
- [7] GitHub Docs, “Building and testing Node.js.” [Online]. Tersedia: <https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-nodejs>.
- [8] GitHub Docs, “Storing and sharing data from a workflow.” [Online]. Tersedia: <https://docs.github.com/en/actions/using-workflows/storing-workflow-data-as-artifacts>.
- [9] Vitest, “Vitest Documentation.” [Online]. Tersedia: <https://vitest.dev/guide/>.
- [10] ESLint, “ESLint Documentation.” [Online]. Tersedia: <https://eslint.org/docs/latest/>.